



SISTEMAS DISTRIBUIDOS

Task1:Communication and middleware

2025-26

SALMA JADIANI

ALEJANDRO FERNÁNDEZ

ÍNDICE

- 1. Introducción**
- 2. Diseño del Sistema**
 - 2.1. Modelos de tickets**
 - 2.2. Arquitecturas**
 - 2.3. Consistencia y Redis**
- 3. Despliegue (AWS Academy)**
- 4. Resultados y gráficas**
- 5. Conclusiones**

1.Introducción

- **Objetivos:**

La gestión de venta de entradas para eventos de alta demanda (conciertos, eventos deportivos, etc.) representa uno de los desafíos más críticos en el diseño de sistemas distribuidos. El problema fundamental radica en la **conurrencia masiva**: miles de usuarios intentando adquirir un inventario limitado de asientos en el mismo segundo.

En esta práctica, se han implementado y analizado dos arquitecturas de comunicación para resolver este escenario:

1. **Comunicación Directa:** Basada en Pyro4 como middleware de RPC síncrono, con workers registrados en un Name Server y un balanceador de carga round-robin en el lado del cliente para distribuir las peticiones.
2. **Comunicación Indirecta:** Basada en un modelo asíncrono utilizando **RabbitMQ** como middleware de mensajería, donde el procesamiento se desacopla de la recepción mediante el uso de *Workers*.

El sistema utiliza **Redis** como motor de persistencia en memoria, aprovechando sus capacidades atómicas para garantizar que el estado de los tickets sea consistente en todo momento.

El desarrollo de este proyecto se ha guiado por los siguientes objetivos principales:

- **Garantizar la Consistencia de Datos:** Asegurar que, bajo ninguna circunstancia, un mismo asiento numerado sea vendido a dos clientes distintos (evitar el *overselling*).
- **Implementar Robustez y Resiliencia:** Diseñar el sistema de comunicación indirecta de modo que sea capaz de sobrevivir a fallos críticos. Esto incluye la persistencia de mensajes en la cola y el manejo de confirmaciones manuales para evitar la pérdida de transacciones si un procesador (*Worker*) se desconecta.
- **Analizar la Escalabilidad:** Evaluar cómo el rendimiento del sistema (medido en operaciones por segundo o *throughput*) mejora de forma dinámica al añadir más nodos de procesamiento.
- **Estudiar el Impacto de la Contención (Hotspots):** Observar y documentar el comportamiento del sistema cuando una gran mayoría de las peticiones se concentran en un subconjunto muy pequeño de recursos (asientos específicos), provocando cuellos de

botella en la base de datos.

- **Comparativa Crítica:** Determinar las ventajas y desventajas de cada modelo de comunicación en función de la carga de trabajo y los requisitos de disponibilidad.

2. Diseño del Sistema

2.1 Modelos de tickets

- **No numerados.**

Este modelo representa eventos donde las entradas no tienen un asiento asignado (ej. pista en un concierto).

- **Lógica de Implementación:** Se utiliza un contador atómico en Redis mediante el comando **INCR**.

```
tickets_vendidos = self.redis.incr("contador_tickets")
```

- **Gestión de Concurrencia:** El sistema comprueba si el valor del contador es inferior al aforo total antes de confirmar la venta.

```
tickets_vendidos <= 20000:
```

- **Rendimiento:** Al no haber colisiones por un recurso específico, la contención es mínima, permitiendo que múltiples Workers incrementen el contador simultáneamente sin bloqueos.
 - Numerados.

Este modelo es propenso a generar **Hotspots**, ya que cada petición solicita un identificador de asiento

- **Lógica de Implementación:** Se utiliza una estructura de clave-valor donde la clave es el identificador del asiento. La reserva se realiza mediante la operación atómica SET NX (Set if Not Exists).

```
reservado = self.redis.set(clave_asiento, cliente_id, nx=True)
```

- **Garantía de Exclusividad:** El uso de NX asegura que solo la primera petición que llegue a Redis consiga escribir su request_id en esa clave. Cualquier intento posterior para el mismo asiento será rechazado por la base de datos, devolviendo un error de "asiento ya vendido".
- **Impacto en el Diseño:** En los benchmarks de alta carga, muchas peticiones compiten por el mismo asiento (Hotspot), lo que obliga a

los Workers a procesar y descartar rápidamente las peticiones duplicadas para mantener la consistencia.

2.2 Arquitecturas

- **Directa (PYRO4)**

En este modelo, un objeto en una máquina (servidor) expone sus métodos para que puedan ser ejecutados por otra máquina (cliente) de forma transparente, como si fueran llamadas locales. Basado en RPC (*Remote Procedure Calls*). Este modelo permite que un cliente invoque métodos de objetos remotos de forma transparente, como si fueran llamadas locales.

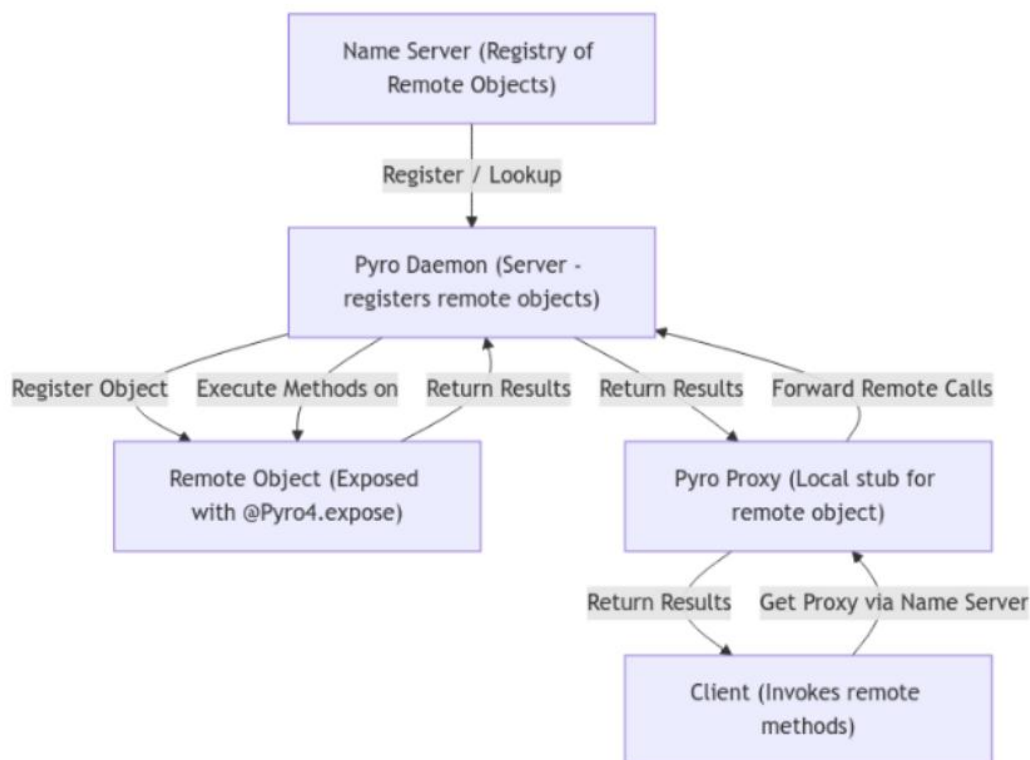


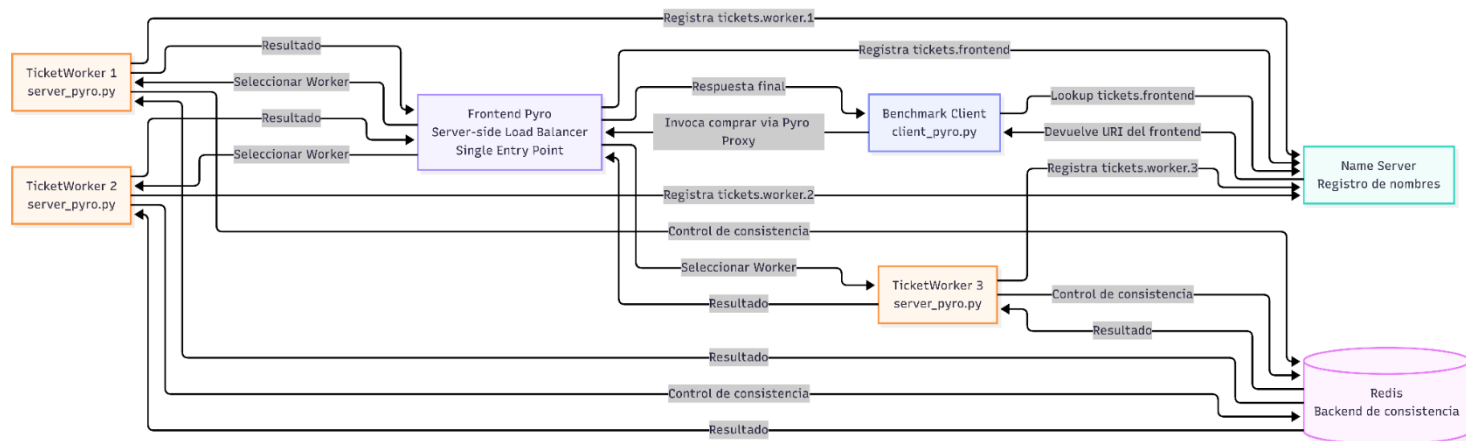
Fig. 1 - Arquitectura de Comunicación Directa (Pyro4)

Los componentes clave definidos en la teoría y aplicados en el proyecto son:

- **Name Server:** servicio encargado de registrar y localizar los objetos remotos.
- **Pyro Proxy:** proxy local que utiliza el cliente para comunicarse con los objetos remotos.
- **Pyro Daemon:** proceso servidor que recibe y gestiona las

peticiones RPC.

- **Remote Objects:** objetos remotos expuestos mediante el decorador `@Pyro4.expose`.



La arquitectura implementada se organiza en varios niveles.

Name Server

El sistema utiliza el Name Server de Pyro4 para desacoplar la ubicación física de los servicios. Tanto el frontend como los distintos workers se registran dinámicamente utilizando identificadores únicos:

- tickets.frontend
- tickets.worker.1
- tickets.worker.2
- tickets.worker.3

Gracias a este mecanismo, es posible añadir o eliminar workers sin necesidad de modificar el cliente.

Frontend Pyro (Single Entry Point)

La arquitectura implementa un único punto de entrada (`tickets.frontend`) que actúa como balanceador de carga del lado del servidor (*Server-side Load Balancer*).

El cliente únicamente consulta el objeto `tickets.frontend` en el Name Server y envía todas las peticiones a dicho frontend. Posteriormente, el frontend obtiene la lista de workers disponibles, selecciona uno mediante un algoritmo *Round Robin* y reenvía la petición RPC al worker correspondiente.

Este enfoque permite desacoplar completamente el cliente de los workers internos del sistema y facilita el escalado dinámico de la arquitectura.

Workers

El archivo `server_pyro.py` implementa la clase `TicketWorker`, marcada con `@Pyro4.expose`.

Cada worker recibe las peticiones RPC enviadas por el frontend, ejecuta la lógica de compra y se comunica con Redis para garantizar la consistencia del sistema y evitar situaciones de sobreventa.

Backend de Consistencia (Redis)

Redis se utiliza como backend de consistencia mediante operaciones atómicas:

- INCR para tickets no numerados.
- SET NX para tickets numerados.

Estas operaciones permiten garantizar:

- ausencia de doble venta,
- consistencia bajo concurrencia,
- idempotencia de peticiones.

Cliente Benchmark

El archivo `client_pyro.py` se encarga de cargar los benchmarks y generar las peticiones concurrentes utilizando múltiples hilos.

El cliente únicamente interactúa con el frontend (`tickets.frontend`),

manteniendo el desacoplamiento respecto a los workers internos del sistema.

Mitigación del Sencillo Point of Failure (SPOF):

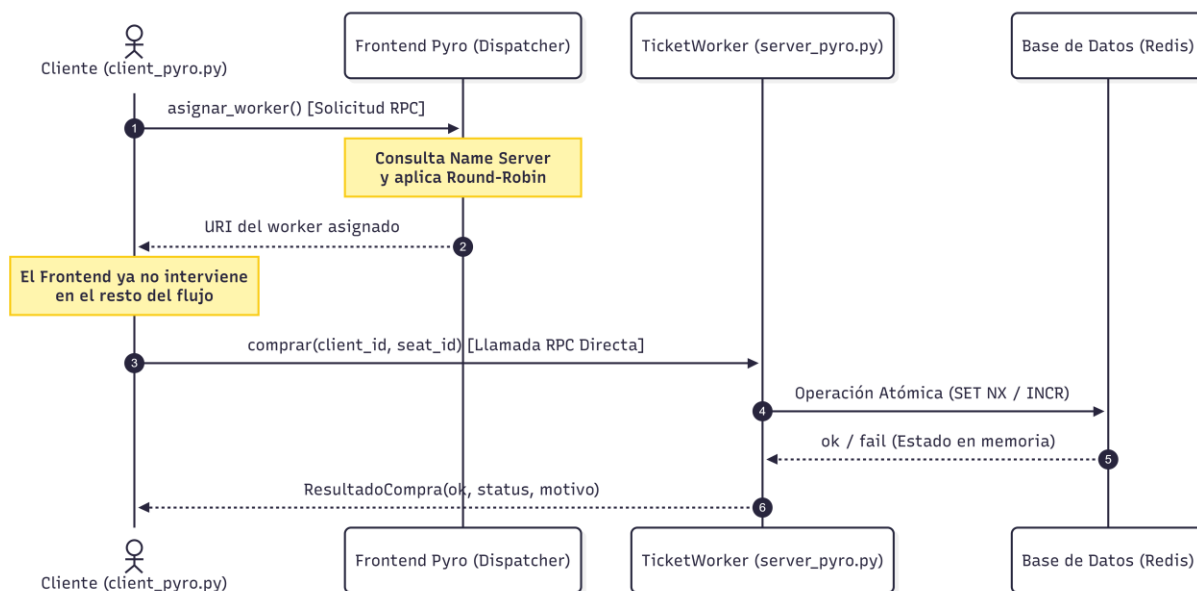
El cliente ya no satura al Frontend con la transferencia de datos de la transacción; le solicita un destino.

El Frontend ejecuta un algoritmo *Round-Robin*, consulta los workers activos en el *Name Server* y le devuelve al cliente la dirección (URI) del worker seleccionado.

Una vez que el cliente obtiene la URI, **la comunicación pasa a ser puramente directa entre Cliente y Worker**, dejando al Frontend completamente libre para atender la siguiente solicitud de asignación.

Ventajas:

1. Si el Frontend se cae *mientras* los workers están procesando o respondiendo al cliente, las compras en curso finalizan con éxito, ya que la conexión síncrona se mantiene directamente entre cada hilo del cliente y su respectivo worker.
2. Al no retener estado (*stateless*) ni intervenir en el viaje de vuelta de los datos, el impacto de una caída del Frontend disminuye drásticamente, permitiendo levantar réplicas idénticas detrás de un balanceador de red sin necesidad de replicar contextos de sesión.



- Indirecta (RabbitMQ)

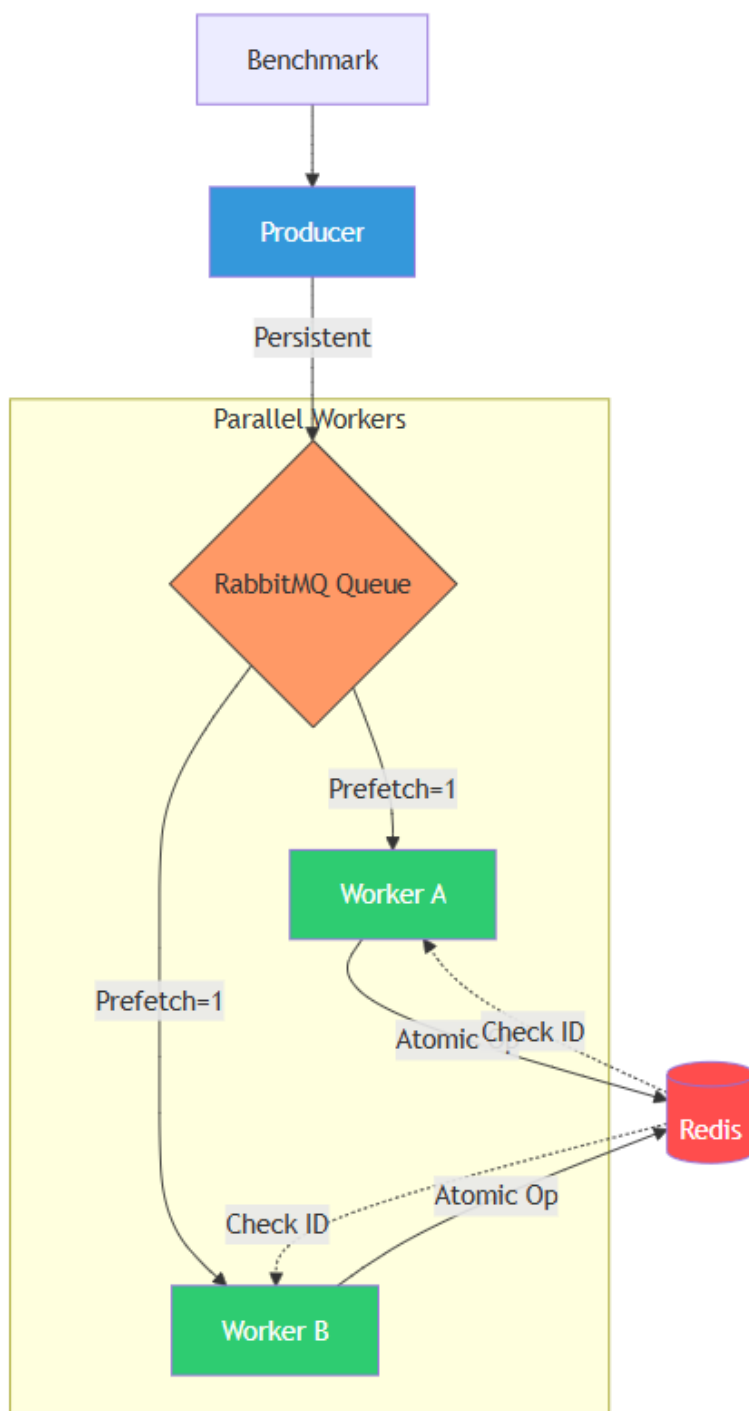


Fig. 2 - Arquitectura de Comunicación Indirecta (RabbitMQ)

En el modelo indirecto un *Producer* desacopla la carga del archivo de *benchmark* del procesamiento real y envía peticiones persistentes a una cola de RabbitMQ “cola_tickets”. El sistema escala horizontalmente

mediante múltiples Workers que, bajo una política de Fair Dispatch (Prefetch=1) y confirmaciones manuales (ACK), consumen mensajes de forma equilibrada, asegurando que ninguna tarea se pierda si un nodo falla. Finalmente, todos los Workers convergen en una base de datos Redis, donde se ejecutan operaciones atómicas (SET NX / INCR) y validaciones de idempotencia mediante el `request_id`, garantizando así la consistencia absoluta del inventario y evitando la sobreventa de tickets incluso en escenarios de alta contención o reintentos de red.

Fire and forget:

El patrón Fire-and-Forget se implementa en `producer.py`. El productor lee el archivo de benchmark línea a línea, construye un mensaje JSON con los datos de la compra y lo publica en la 'cola_tickets' de RabbitMQ con `delivery_mode=2` (persistente).

Una vez publicado el mensaje, el productor continúa con la siguiente línea sin esperar ninguna respuesta del procesamiento.

El desacoplamiento es total: el productor desconoce cuántos Workers hay, o el estado de la petición. RabbitMQ actúa como buffer almacenando los mensajes de forma persistente en disco hasta que los Workers los consuman desacoblando al cliente del proceso.

2.3 Consistencia, Redis.

Para garantizar la integridad del inventario, el sistema delega la exclusividad de las ventas a Redis, actuando frente a los múltiples Workers.

En el caso de asientos numerados, el código utiliza la instrucción `set(clave, valor, nx=True)`, donde el parámetro NX asegura que la reserva solo sea exitosa si el asiento no ha sido ocupado previamente por otro proceso.

Para los tickets no numerados, se emplea la función `incr(contador)`, que permite gestionar el aforo de forma segura mediante incrementos atómicos que evitan condiciones de carrera.

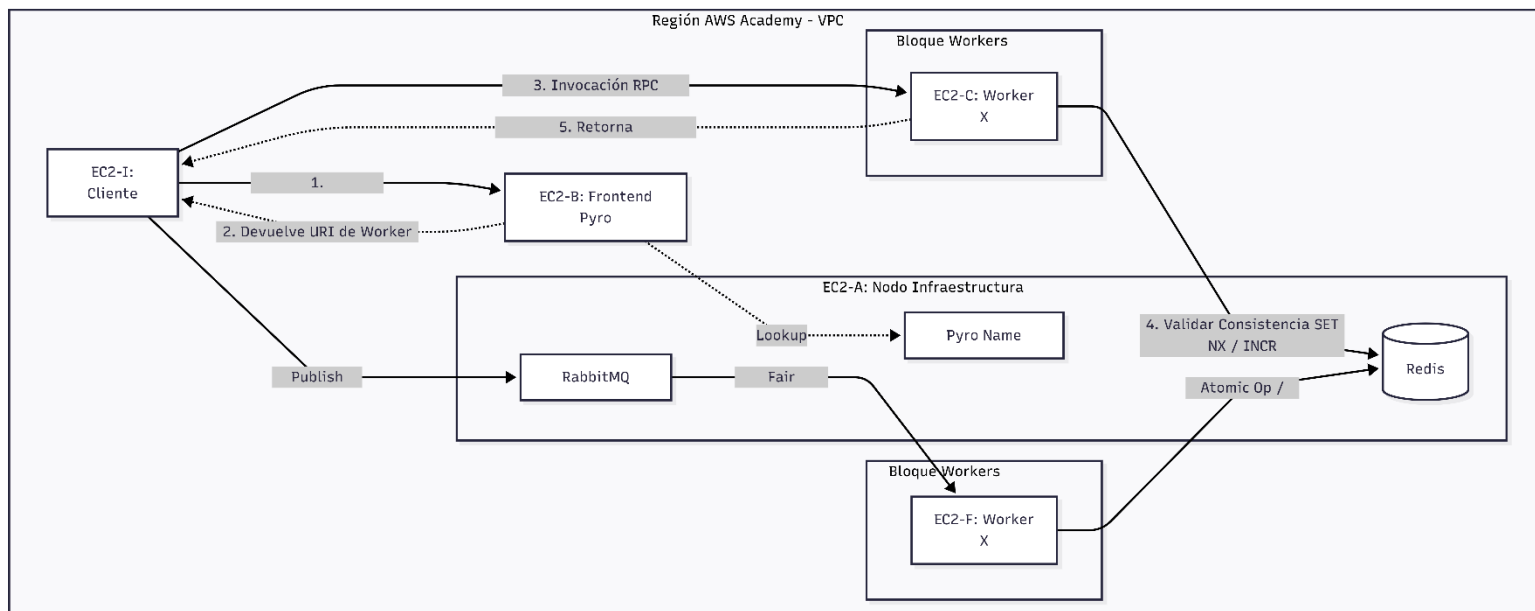
Finalmente, la idempotencia se asegura mediante un registro de `request_id` en Redis: antes de confirmar cualquier operación, el Worker verifica si dicho identificador ya existe, evitando que reintentos de red de RabbitMQ provoquen ventas duplicadas y garantizando que el estado final de la base de datos sea siempre consistente.

3. Despliegue (AWS)

El sistema se ha desplegado en AWS Academy Learner Lab, **usando 9 instancias EC2** (Amazon Linux 2023) interconectadas por red privada. Cada instancia cumple un rol fijo dentro de la arquitectura:

Instancia	Rol	IP privada
EC2-A-infra	Redis, RabbitMQ, Pyro Name Server	172.31.33.235
EC2-B-frontend	Frontend Pyro (dispatcher)	172.31.17.238
EC2-C/D/E-worker-directo	Workers comunicación directa (1-3)	172.31.21.3 / 18.167 / 19.71
EC2-F/G/H-worker-indirecto	Workers comunicación indirecta (1-3)	172.31.26.72 / 20.248 / 25.111
EC2-I-cliente	Cliente, ejecuta los benchmarks	172.31.16.31

A diferencia del despliegue inicial en dos máquinas, este modelo separa cada componente en su propia instancia, lo que permite escalar el número de workers de forma independiente para cada arquitectura y



aislar el efecto de cada nodo en las métricas de rendimiento. Véase el diagrama:

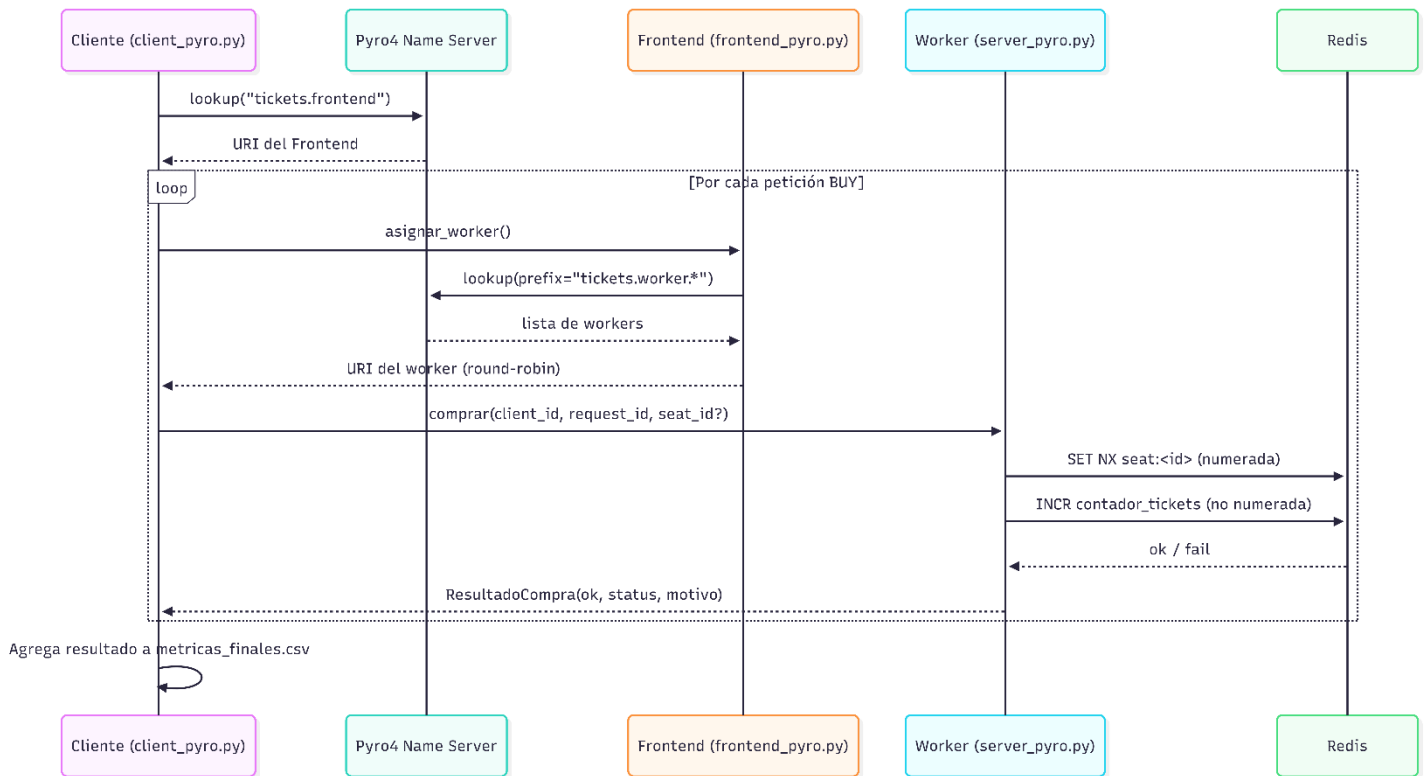
Flujo ejecución:**Comunicación directa (Pyro)**

1. Se inicia el Name Server en EC2-A.
2. Se lanzan entre 1 y 3 workers en EC2-C/D/E, que se registran dinámicamente.
3. Se lanza el frontend en EC2-B, que descubre los workers activos vía Name Server.
4. El cliente en EC2-I envía las peticiones al frontend, que las reparte en round-robin.

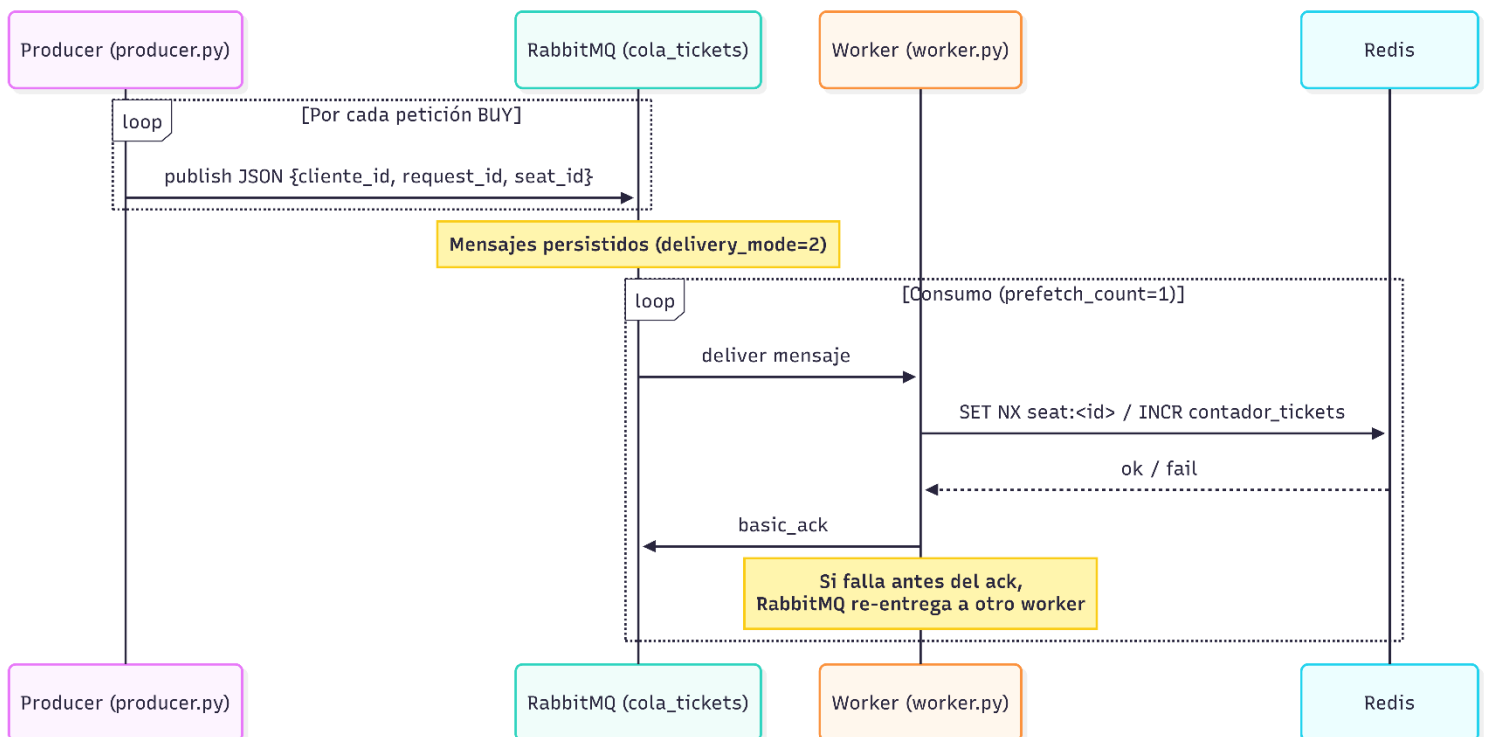
Comunicación indirecta (RabbitMQ)

1. El broker RabbitMQ corre en EC2-A.
2. Se lanzan entre 1 y 3 workers consumidores en EC2-F/G/H.
3. El cliente en EC2-I publica las peticiones en la cola mediante el producer.

Comunicación directa (Pyro)



Comunicación indirecta (RabbitMQ)

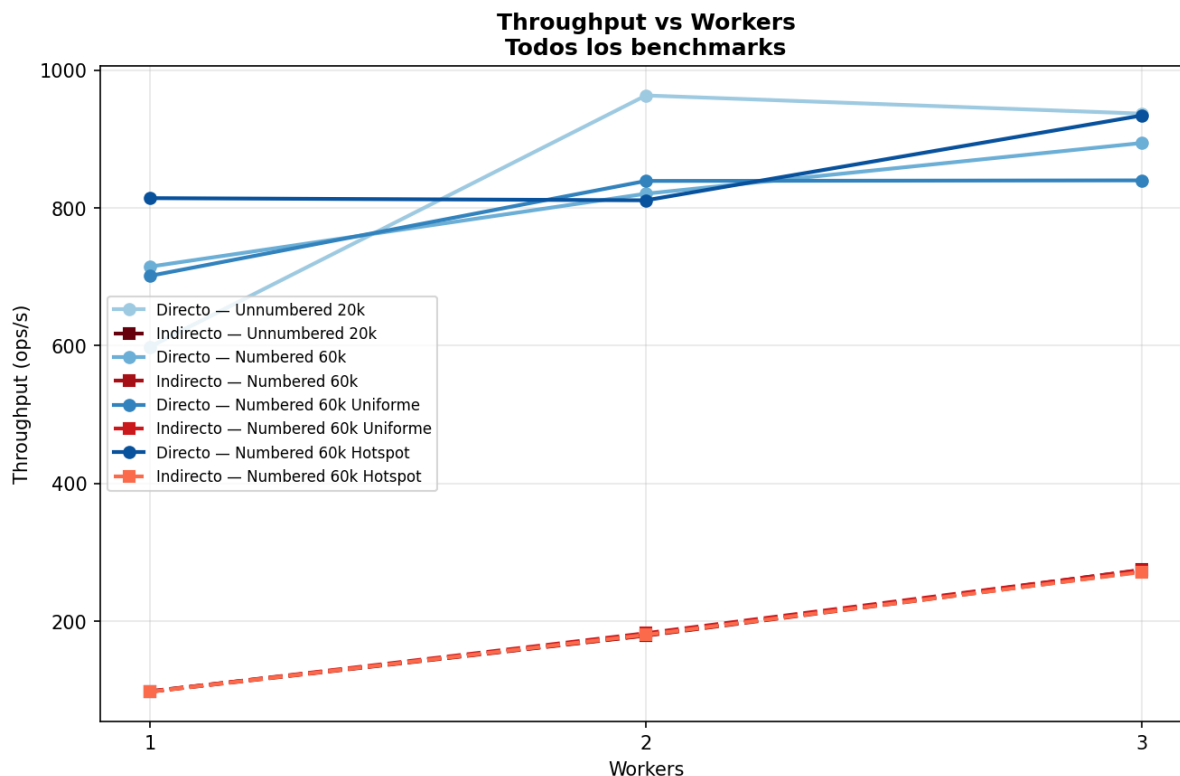


4. Resultados y gráficas

modelo	benchmark	workers	tiempo_envio	tiempo_total	throughput_total	success	fail
indirecto	bm_uniforme.txt	3	9.82	219.03	273.93	19022	40978
indirecto	bm_hotspot.txt	3	9.45	221.20	271.25	10008	49992
indirecto	benchmark_numbered_60000.txt	3	4.16	95.62	271.89	20000	5997
indirecto	benchmark_unnumbered_20000.txt	3	3.15	72.9	274.35	20000	0
indirecto	bm_uniforme.txt	2	9.53	329.17	182.28	19022	40978
indirecto	bm_hotspot.txt	2	9.6	333.5	179.91	10008	49992
indirecto	benchmark_numbered_60000.txt	2	4.38	145.41	178.78	20000	5997
indirecto	benchmark_unnumbered_20000.txt	2	3.41	111.59	179.23	20000	0
indirecto	bm_uniforme.txt	1	10.09	615.34	97.51	19022	40978
indirecto	bm_hotspot.txt	1	9.91	616.66	97.3	10008	49992
indirecto	benchmark_numbered_60000.txt	1	4.23	267.17	97.3	20000	5997
indirecto	benchmark_unnumbered_20000.txt	1	3.31	204.11	97.99	20000	0
directo	bm_uniforme.txt	3		71.43	839.97	19022	40978
directo	bm_hotspot.txt	3		64.22	934.27	10008	49992
directo	benchmark_numbered_60000.txt	3		29.07	894.29	20000	5997
directo	benchmark_unnumbered_20000.txt	3		21.35	936.88	20000	0
directo	bm_uniforme.txt	2		71.5	839.17	19022	40978
directo	bm_hotspot.txt	2		73.98	811.05	10008	49992
directo	benchmark_numbered_60000.txt	2		31.68	820.64	20000	5997
directo	benchmark_unnumbered_20000.txt	2		20.76	963.31	20000	0
directo	bm_uniforme.txt	1		85.56	701.27	19022	40978
directo	bm_hotspot.txt	1		73.69	814.21	10008	49992
directo	benchmark_numbered_60000.txt	1		36.38	714.69	20000	5997
directo	benchmark_numbered_60000.txt	1		33.43	598.24	20000	0

A continuación se presentan las gráficas de rendimiento obtenidas para cada benchmark en ambas arquitecturas. Todas las gráficas enfrentan el throughput (ops/seg) y el tiempo total (segundos) frente al número de Workers activos (1, 2 y 3 para indirecto; 1, 2 y 3 para directo), permitiendo observar la escalabilidad de cada modelo. Los datos numéricos completos se recogen en la Tabla.

• Throughput vs workers



Arquitectura	Mínimo (1 Worker)	Máximo (3 Workers)	Comportamiento
Directo	598.24	963.31	Alto throughput, saturación temprana.
Indirecto	97.30	274.35	Bajo throughput, escalabilidad lineal.

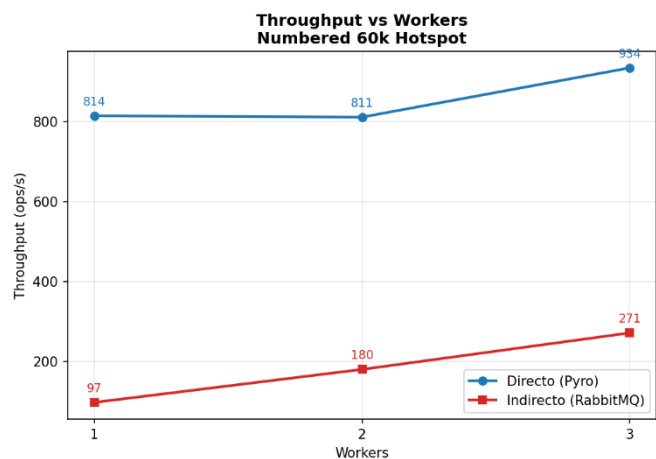
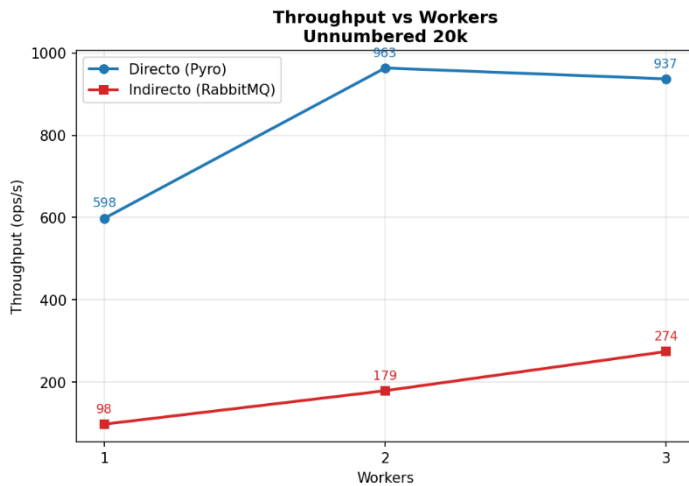
El **modelo indirecto demuestra una escalabilidad horizontal casi perfecta** donde por cada worker agregado el número de operaciones incrementa proporcionalmente.

Al pasar de 1 a 3 workers, el rendimiento prácticamente se duplica y triplica de forma matemática de 97 a 179 a 274. Por el contrario, el **modelo directo experimenta una progresión irregular y un estancamiento prematuro** entre el segundo y tercer worker.

También es notable el gran espacio entre el rendimiento de los dos modelos. Esto ocurre porque en Rabbit hay más latencia por la red y la estructura intermedia (cola donde van las peticiones) mientras que en el modelo directo la comunicación es directa con el cliente.

Por lo referente a como escalan los sistemas; en redis los workers son independientes entre si mientras que en comunicación directa la asignación inicial de workers está centralizada en el frontend.

Máximo y mínimo en comunicación directa:



A- Con un solo worker el directo todavía no satura su capacidad de envío, así que el salto a 2 workers casi duplica el throughput; con 3 el cuello de botella pasa al frontend único, que como dispatcher ya no consigue asignar peticiones más rápido de lo que tarda en decidir a qué worker enviarlas.

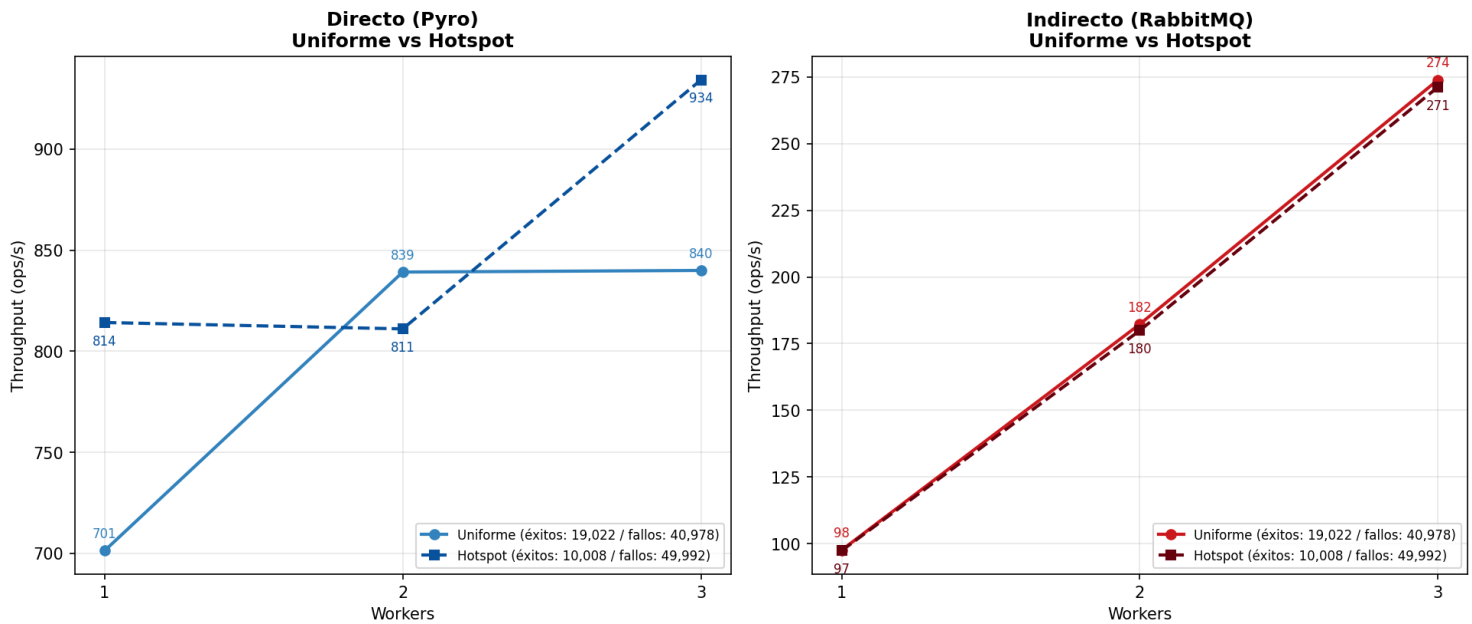
B- Entre 1 y 2 workers el throughput se estanca porque ambos compiten constantemente por el mismo puñado de asientos calientes en Redis, y solo con el tercer worker en el sistema hay suficiente capacidad en paralelo para que el frontend reparta sin generar cola de espera.

Resumen:

- **Indirecto: escalabilidad casi lineal.**
- **Directo: progresión irregular con estancamiento prematuro entre el segundo y tercer worker.**
- **Gran diferencia de rendimiento entre ambos modelos por la latencia de red y la cola intermedia de RabbitMQ frente a la comunicación directa con el cliente.**
- **Los workers indirectos son independientes entre sí; en el directo, la asignación pasa siempre por el frontend centralizado.**

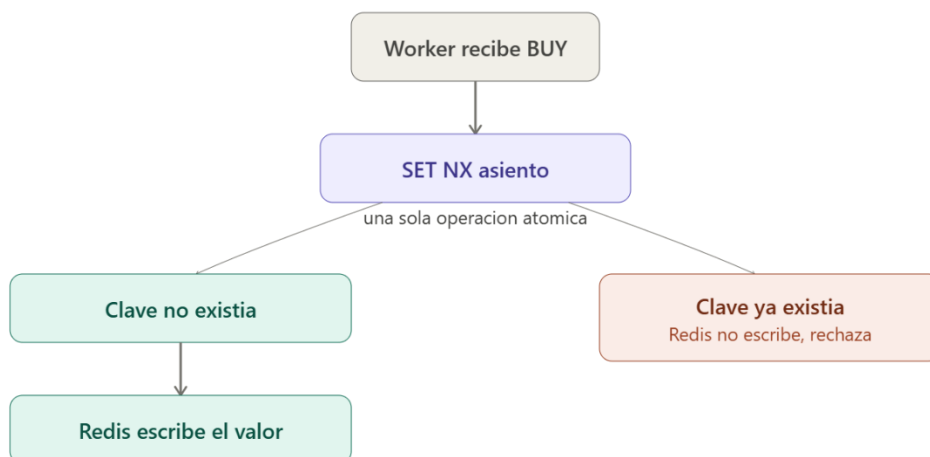
- Impacto de la contención – Hotspot

Impacto de la contención (hotspot) por arquitectura



En ambos gráficos se observa que para 3 workers tanto el benchmark con contención como el de distribución enorme alcanzan a casi el mismo número de operaciones por segundo.

En la comunicación directa con 1 worker el hotspot supera al uniforme (814 vs 714 ops/s). debido a que la mayoría de las peticiones del hotspot caen sobre asientos ya ocupados, y Redis resuelve un rechazo (la clave ya existe) más rápido que una escritura real de un asiento libre.



Al concentrarse las peticiones en muy pocos asientos, el sistema pasa más tiempo descartando peticiones baratas que completando reservas nuevas (operaciones rápidas), lo que eleva ligeramente el throughput total ya que rechazar una BUY también es resolver una petición.

Con 2 workers el patrón se invierte ligeramente el benchmark de contención se estanca, debido a que 2 workers compitiendo simultáneamente por los mismos asientos calientes pueden generar algo de contención real en Redis, sin que el paralelismo añadido sea aún suficiente para compensarla.

Sin embargo, dado que la diferencia es pequeña (3.4%) y no sigue una tendencia consistente con los otros puntos, también podría tratarse de variabilidad de red entre ejecuciones independientes en AWS.

Con 3 workers el hotspot se dispara hasta 934 ops/s (+15% respecto a 2 workers), mientras el uniforme permanece prácticamente plano (839 a 840). Esto es coherente con la naturaleza de cada carga: en el hotspot, el trabajo añadido por el tercer worker es mayormente descartar peticiones repetidas sobre asientos ya vendidos, una operación barata que escala bien en paralelo; en el uniforme, en cambio, la mayoría de las peticiones requieren completar una escritura real en Redis, un trabajo más costoso que no se beneficia tanto de añadir un worker más mientras el frontend siga siendo el único punto de entrada.

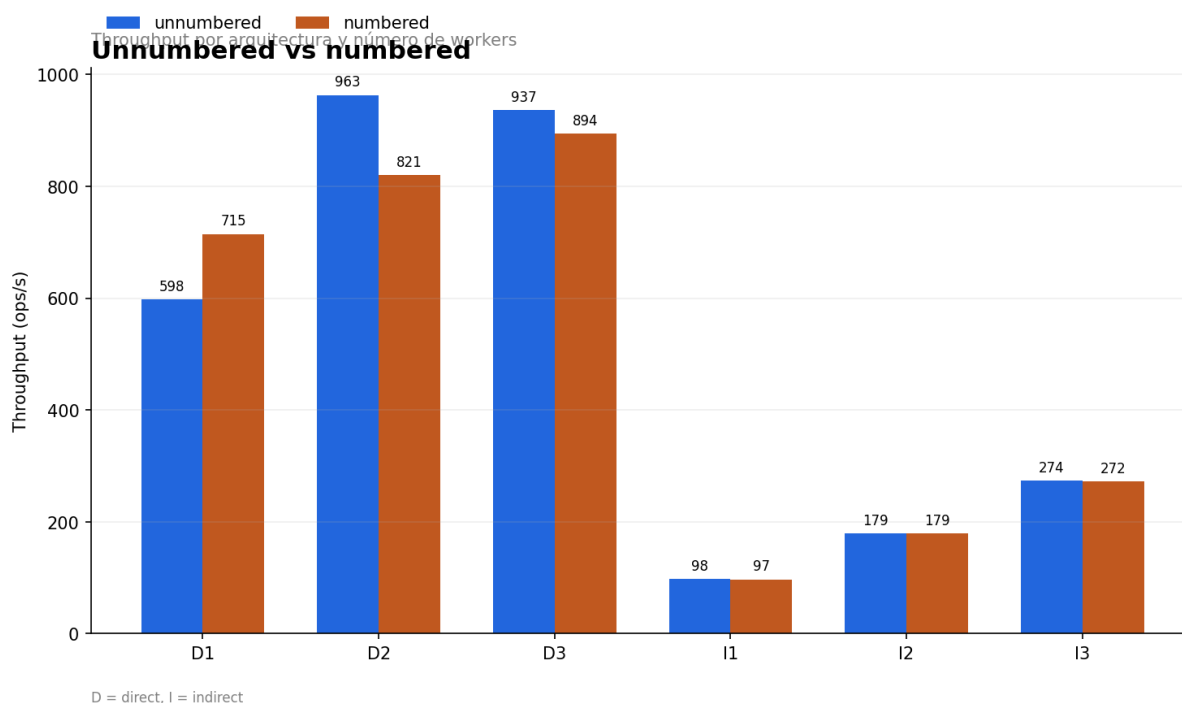
En el indirecto, el cuello de botella no está en cuánto cuesta cada operación individual en Redis sino en el ritmo al que cada worker consume mensajes de la cola, uno a uno. Como ese ritmo de consumo es el mismo independientemente de si la operación termina en éxito o en rechazo, la pequeña diferencia de coste entre escribir y rechazar en Redis queda totalmente frente al tiempo dominante: el propio procesamiento secuencial del worker sacando mensajes de la cola.

Resumen:

- **El hotspot no degrada el rendimiento en ninguna arquitectura.**
- **Directo: el hotspot mejora ligeramente el throughput porque rechazar es más barato que escribir en Redis, y ese ahorro se nota más con más workers.**
- **La pequeña inversión con 2 workers es probablemente ruido de medición, no una tendencia real.**

- Indirecto: el ritmo de consumo de la cola manda (con o sin hotspot);
- Redis absorbe bien la contención; el rendimiento depende del mecanismo de comunicación, no del patrón de acceso a los datos.

• Numbered vs Unnumbered

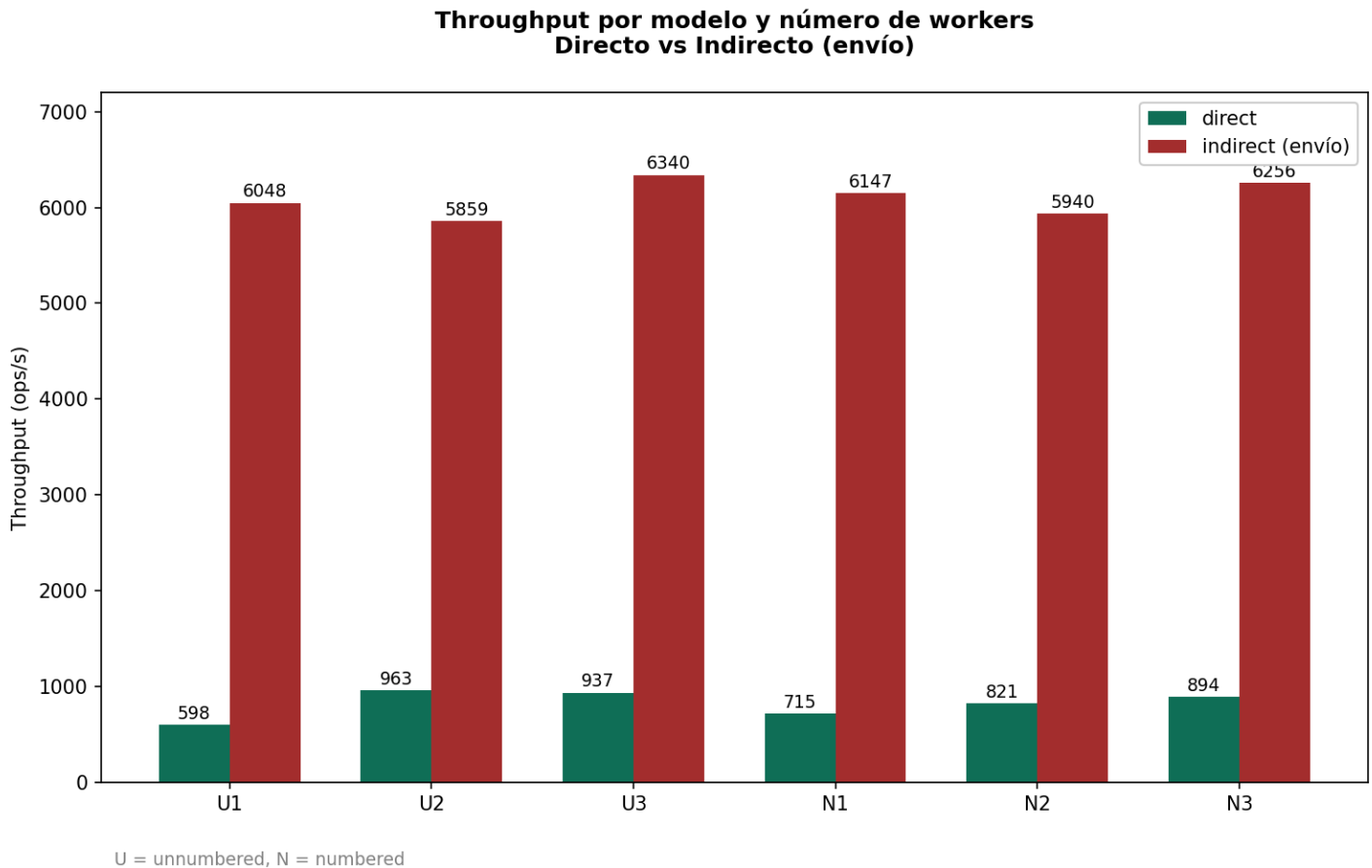


Con 1 worker, numbered supera a unnumbered (715 vs 598 ops/s). Una causa probable es que ambos benchmarks no tienen el mismo volumen de peticiones (**25997 en numbered** frente a **20000 en unnumbered**): con menos de un minuto de ejecución total, el coste fijo de arranque del sistema (conexión al frontend, Redis) pesa proporcionalmente más sobre un volumen menor de operaciones, lo que penaliza el throughput medio de unnumbered en esta comparación concreta. Dado que ambos provienen de archivos de benchmark distintos en volumen y patrón de carga, esta diferencia no debe interpretarse como una propiedad general del modelo de ticket.

Resumen:

- Cada modelo sirve para algo distinto: unnumbered para medir cuánto aforo queda, numbered para controlar quién se queda con cada asiento.

- **Direct Vs Indirect (tiempo envío y tiempo e2e)**

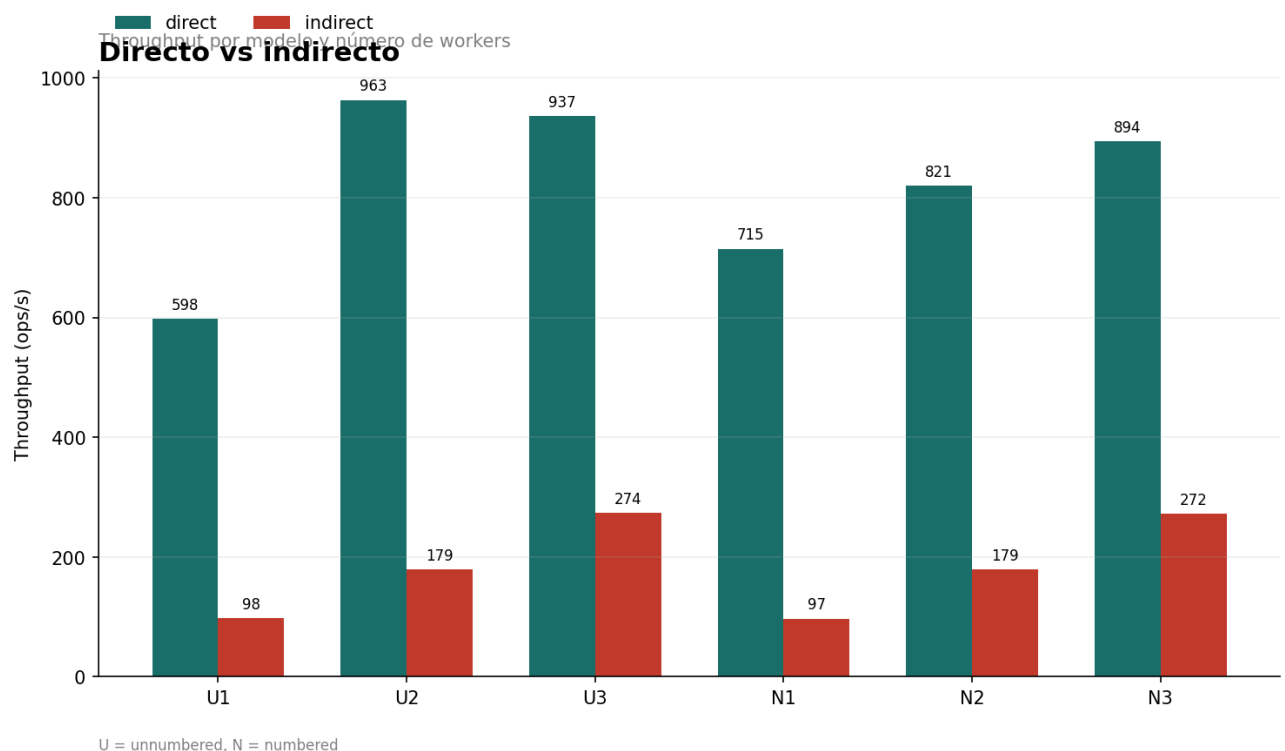


Tomando en consideración los tiempos de envío del modelo indirecto y el tiempo total que requiere el sincronismo del modelo directo a la hora de calcular throughput se observa que el rendimiento es mucho superior en el indirect en todos los casos.

Esto se debe a que este **modelo indirecto permite un desacoplamiento total entre el ritmo del envío y el ritmo del procesamiento**. El ritmo de envío es el tiempo en el cual los clientes encolan las peticiones y se olvidan de su procesamiento. El mismo nombre fire-and-forget indica esta característica. Cabe mencionar que este modelo permite **absorber ráfagas en la cola intermedia y facilita la escalabilidad progresiva** de los sistemas en función del número de workers.

A continuación se observa la misma gráfica tomando en consideración el tiempo total en cada modelo de comunicación. En el modelo directo estos tiempos se calculan client-side y en el modelo indirecto el tiempo lo calcula el servidor, una vez el worker resuelve la petición.

En la siguiente gráfica para el tiempo del modelo indirecto se ha tomado Tiempo Fire and forget + tiempo procesamiento (server side). Nótese entonces que el **modelo directo** supera en rendimiento al indirecto debido a su **Menor latencia extremo a extremo, menor sobrecarga de una cola intermedia (RabbitMQ) ofreciendo un mejor throughput en todas las mediciones del experimento.**



Conclusiones:

La **comunicación directa con Pyro4 ofrece un modelo sencillo** de naturaleza síncrona que la hace vulnerable a fallos y poco eficiente cuando el frontend, como único punto de entrada, se convierte en el cuello de botella. En el despliegue en AWS Academy, el Name Server y el Frontend actúan como punto único de entrada en su propia instancia, mientras los Workers se distribuyen en instancias independientes, pero el cliente siempre queda bloqueado esperando la respuesta RPC.

La **comunicación indirecta con RabbitMQ demuestra mayor resiliencia y mejor escalabilidad** proporcional al añadir workers, especialmente notable en el throughput de procesamiento, que prácticamente se triplica al pasar de 1 a 3 workers. Sin embargo, en el rango de 1 a 3 workers probado, su throughput absoluto queda muy por debajo del modelo directo (entre 97 y 274 ops/s frente a 598 y 963 ops/s), ya que el cuello de botella no está en Redis —que resuelve la contención de forma eficiente gracias a operaciones atómicas como SET NX— sino en el ritmo de consumo secuencial de la cola por parte de cada worker. En el despliegue en AWS, RabbitMQ y Redis residen en la misma instancia de infraestructura, los Workers consumidores se ejecutan en instancias independientes, y el Producer (cliente) envía las peticiones de forma asíncrona sin bloquearse.

En conclusión, **no existe una arquitectura universalmente mejor**: la elección depende de qué se priorice. La arquitectura directa (Pyro4) resulta más adecuada cuando el rendimiento absoluto con pocos recursos es prioritario, gracias a su camino de ejecución más corto y la ausencia de overhead de cola. La arquitectura indirecta (RabbitMQ) resulta más adecuada en escenarios con mayor disponibilidad de workers o donde la resiliencia (persistencia de mensajes, ACKs, desacoplamiento productor-consumidor) pesa más que la velocidad bruta, ya que su capacidad de escalado proporcional sugiere que con suficientes workers podría llegar a igualar o superar al modelo directo, algo que este estudio no llega a demostrar dentro del rango de 1 a 3 workers evaluado.